

Processes versus people: How should agile software development maturity be defined?



Rafaela Mantovani Fontana^{a,b,*}, Isabela Mantovani Fontana^c,
Paula Andrea da Rosa Garbuio^a, Sheila Reinehr^a, Andreia Malucelli^a

^a Pontifical Catholic University of Paraná (PUCPR), R. Imaculada Conceição, 1155, Prado Velho, 80215-901 Curitiba, PR, Brazil

^b Federal University of Paraná (UFPR), R. Dr. Alcides Vieira Arcoverde, 1225, Jd. das Américas, 81520-260 Curitiba, PR, Brazil

^c Polytechnic School of the University of São Paulo (USP), Av. Prof Almeida Prado, 128, Cidade Universitária, PO Box 61548, 05508-070 São Paulo, SP, Brazil

ARTICLE INFO

Article history:

Received 5 September 2013

Received in revised form 15 July 2014

Accepted 15 July 2014

Available online 24 July 2014

Keywords:

Maturity

Agile software development

Software process improvement

ABSTRACT

Maturity in software development is currently defined by models such as CMMI-DEV and ISO/IEC 15504, which emphasize the need to manage, establish, measure and optimize processes. Teams that develop software using these models are guided by defined, detailed processes. However, an increasing number of teams have been implementing agile software development methods that focus on people rather than processes. What, then, is maturity for these agile teams that focus less on detailed, defined processes? This is the question we sought to answer in this study. To this end, we asked agile practitioners about their perception of the maturity level of a number of practices and how they defined maturity in agile software development. We used cluster analysis to analyze quantitative data and triangulated the results with content analysis of the qualitative data. We then proposed a new definition for agile software development maturity. The findings show that practitioners do not see maturity in agile software development as process definition or quantitative management capabilities. Rather, agile maturity means fostering more subjective capabilities, such as collaboration, communication, commitment, care, sharing and self-organization.

© 2014 Elsevier Inc. All rights reserved.

1. Introduction

A number of agile software development methods with different properties and applications have been proposed in recent years. All are people-focused, communication-oriented, flexible, speedy, lean, responsive and learning-oriented (Qumer and Henderson-Sellers, 2008b). These characteristics, disclosed in the Agile Manifesto (Beck et al., 2001), have brought changes to software engineering as never before, including a variety of new software methods, tools, techniques and best practices (Dingsøyr et al., 2012).

These new methods encompass different aspects of software development. Scrum and Internet-speed Development (ISD), for example, emphasize project management practices; Agile

Modeling, Extreme Programming (XP) and Pragmatic Programming cover aspects related to software development; and others, such as Adaptive Software Development, Crystal Family, Dynamic Systems Development Model (DSDM) and Feature-driven Development (FDD), support both project management and software development processes (Abrahamsson et al., 2003). Scrum is one of the most widely adopted methods, but usually with significant tailoring, i.e., teams usually adapt practices to their own contexts (Bustard et al., 2013).

One specific context in which agile methods must be adapted is when they are implemented along with software process improvement (SPI) initiatives (Lukasiewicz and Miler, 2012; Baker, 2006; Jakobsen and Johnson, 2008; Sutherland et al., 2007; Cohan and Glazer, 2009; Anderson, 2005; Paulk, 2001). These SPI initiatives are based on the ISO/IEC 15504 (ISO/IEC, 2004) international standard or the Capability Maturity Model Integration for Development (CMMI-DEV), which defines a model that focuses on process definition and control to provide stability and maximize productivity in software development (SEI, 2010). CMMI-DEV requires that processes be formally defined and controlled, which is not usual practice in agile software development environments. Hence, there is a need to adapt agile practices to suit CMMI-DEV requirements.

* Corresponding author at: Federal University of Paraná (UFPR), R. Dr. Alcides Vieira Arcoverde, 1225, Jd. das Américas, 81520-260 Curitiba, PR, Brazil. Tel.: +55 41 33614905.

E-mail addresses: rafaela.fontana@ufpr.br (R.M. Fontana), isabelafontana@usp.br (I.M. Fontana), paula.garbuio@pucpr.br (P.A. da Rosa Garbuio), sheila.reinehr@pucpr.br (S. Reinehr), malu@ppgia.pucpr.br (A. Malucelli).

This need for adaptation brings into focus the fact that the current concept of maturity defined by CMMI-DEV cannot be directly applied to agile methods: it is not possible to maintain agility at higher maturity levels (Lukasiewicz and Miler, 2012; Paulk, 2001). SPI initiatives based on CMMI-DEV or ISO/IEC 15504 focus on improving software productivity rather than on delivering higher value to stakeholders (Boehm and Turner, 2003), which highlights differences between the principles behind SPI initiatives and those behind agile methods (Bhasin, 2012; Siakas and Siakas, 2007). We believe that these differences encourage thinking about other forms of software development maturity; otherwise, agile teams could never achieve maturity without shifting their focus from people to processes.

This paper therefore describes an exploratory study that seeks to discover whether agile maturity is the same as maturity defined by current SPI models. The question we posed was: *how do agile software development practitioners define maturity?* We conducted a survey among Brazilian agile software development practitioners to identify to what extent they classify certain practices as mature and how they define maturity. We applied statistical cluster analysis and content analysis to analyze the data.

In Section 2 we discuss how maturity in agile software development has been described in the literature. Section 3 describes the research approach, Sections 4 and 5 show the results of the data analysis, and Sections 6 and 7 report our understanding of the findings and conclusions.

2. Maturity in agile methods

Maturity models are instruments used to rate capabilities, and based on this rating, initiatives can be implemented to improve the maturity of an element—a person, an object or a social system (Kohlegger et al., 2009). In each model there is thus an underlying concept of the maturity of an element. According to Maier et al. (2012), current maturity models in the field of software engineering and elsewhere have been defined mainly in terms of structured and measurable processes, i.e., maturity is defined as the degree to which processes are institutionalized and effective.

CMMI-DEV and ISO/IEC 15504 share this underlying concept of maturity. CMMI-DEV specifies two types of improvement paths for an organizational unit to become mature: a staged path, which defines multiple process areas for the unit to improve and progress to higher maturity levels; and a continuous one, where individual process areas improve and progress to higher capability levels (SEI, 2010). Process capability, important in both forms of improvement, is a characterization of the ability of a process to achieve current or projected business goals (ISO/IEC, 2004).

The relevance of these improvement guides and the increasing interest in the adoption of agile methods has led a number of researchers to propose ways of combining CMMI-DEV and an agile method (Scrum or XP) to respond to the needs of organizations that seek maturity and agility (Lukasiewicz and Miler, 2012). In accordance with these studies, the last version of CMMI-DEV included comments about agile practices in some of its process areas (Buglione, 2011; SEI, 2010).

The underlying ideas involved in the combining of agile methods and SPI models are:

- that agile processes are applicable to mature organizations (Buglione, 2011);
- that agile methods can be formalized by CMMI-DEV (Baker, 2006);
- that agile thinking guarantees that processes are implemented efficiently while responding to changes, and CMMI-DEV

guarantees that all relevant processes are considered with appropriate discipline (Jakobsen and Johnson, 2008);

- that CMMI-DEV should be used to help organizations institutionalize agile methods, and that agile value is only obtained through discipline (Sutherland et al., 2007);
- that CMMI-DEV brings to agile methods a systematic and quantitative approach to conducting projects using well-known processes (Cohan and Glazer, 2009).

These ideas show that software development teams can obtain real benefits from the combination of agile methods and SPI models. However, regardless of the benefits, the cases in the literature (Lukasiewicz and Miler, 2012; Baker, 2006; Jakobsen and Johnson, 2008; Sutherland et al., 2007; Cohan and Glazer, 2009; Anderson, 2005) always describe how CMMI-DEV can be adapted to agile methods or vice versa.

These adaptations are made because the two approaches do not match: agile methods focus on people, and process improvements on processes. Now that practitioners realize that agile methods require new definitions for maturity, a number of agile maturity models have been proposed (Buglione, 2011; Schweigert et al., 2012). To be suitable for agile environments, these models should be inexpensive, fast and easy to understand, should produce short management reports and should provide relevant drivers and best practices for a road map to maturity (Buglione, 2011).

Next, in Section 2.1, we present four stage-based agile maturity models. These define how agility evolves in a team or organization, usually based on a set of characteristics that may be present. Other related approaches propose factors for measuring agility that we found useful for the goals of this study and are presented later, in Section 2.2.

2.1. Stage-based models

Packlick (2007) proposed an Agile Maturity Map (AMM) that was created to increase quality in agile development. The model was developed at Sabre Airline Solutions and was based on goals rather than processes, unlike other maturity models. The author observed that teams were motivated to find their own ways to achieve their goals. His model defines the following five maturity levels: awareness, transformation, breakthrough, optimization and mentoring.

Sidky et al. (2007) proposed a structured model for adopting agile software development processes. Their framework includes a measurement of agile potential and a four-stage process to introduce agile practices into an organization. To measure agility, the authors proposed a five-level scale: collaborative, evolutionary, effective, adaptive and encompassing. For each level, agile practices should be implemented to address one or more agile principles. To adopt agile methods, the authors proposed a four-stage process that begins by identifying discontinuing factors, is followed by a project level assessment and organizational readiness assessment and finishes with a reconciliation.

Also based on agile adoption, the model proposed by Qumer and Henderson-Sellers (2008a), the Agile Adoption and Improvement Model (AAIM), describes six maturity levels: Agile Infancy, focusing on speed, flexibility and responsiveness; Agile Initial, in which the team is communication-oriented; Agile Realization, in which the focus is on executable artifacts; Agile Value, whose focus is on valuing people and not ignoring processes and tools; Agile Smart, in which priority is learning; and lastly, Agile Progress, which focuses on lean production and remaining agile (Qumer and Henderson-Sellers, 2008a).

The fourth stage-based maturity model was proposed by Patel and Ramachandran (2009). They introduced a five-level model called the Agile Maturity Model (AMM), in which agile practices

evolve from ad hoc to continuous improvement based on agile principles. Each level has a predefined goal to help the development of improvement activities. The levels are Initial, Explored, Defined, Improved and Sustained. The structure of the model is very similar to that of CMMI-DEV (Patel and Ramachandran, 2009).

CMMI-DEV states that “a focus on processes provides the infrastructure and stability necessary to deal with an ever-changing world and to maximize the productivity of people and the use of technology to be competitive” (SEI, 2010, p. 4). The CMMI model defines five maturity stages through which processes evolve: initial, managed, defined, quantitatively managed and, finally, optimizing. In other words, the definition, measurement and control of processes must increase if maturity is to be achieved (SEI, 2010).

It can be seen that the perspective in the four Agile Maturity Models described above is different from that of CMMI-DEV and ISO/IEC 15504. Although processes are defined, the focus is on improving agility. As agility is gradually established, the focus on people, learning and sustaining agility is maintained at the highest maturity levels, showing that maturity here is not achieved exclusively by means of disciplined and controlled processes, as defined by CMMI-DEV and ISO/IEC 15504. As our interest lies in the definition of maturity, we have summarized the definition of the highest level of maturity for each model in Table 1.

2.2. Related approaches

When researching the literature on maturity or process improvement in agile methods, we found some other studies that describe different approaches. We refer to them as “related approaches” because they do not specifically define stages, but identify factors, practices and characteristics that affect agile adoption or allow agility to be measured. We used these studies as references for our measurement instrument, as explained in Section 3.

An initial measure of agility can be obtained from Boehm and Turner’s agility and discipline method (Boehm and Turner, 2003), in which five factors associated with agile and plan-driven characteristics were identified. These factors (criticality, size, culture, dynamism and personnel) allow an assessment of the extent to which an organization is ready for an agile or plan-driven method.

This measure was used by Layman et al. (2004) in a case study at Sabre Airline Solutions. They assessed whether the company would be better suited to a plan-driven or agile method and collected data to investigate agile adoption. The data collection was based on the Extreme Programming Evaluation framework, which has three parts: XP Context Factors, XP Adherence Metrics and XP Outcome Measures. The first part records project context information (team size, team experience, etc.), the second provides a measurement of how far XP practices are adopted and the third provides a way to report success or failure in the adoption of XP practices (Williams et al., 2004).

Also investigating agile adoption, Abbas et al. (2010) identified fifty-eight practices and their correlations with the success of the project. They applied factor analysis to the resulting matrix and identified fifteen factors: architecture modeling, traditional analysis, process/governance, database practice, communication practice and white board, agile quality assurance, communication (team), code analysis and inspection, revision and lightweight testing, architecture and configuration, traditional quality assurance, code standards, lightweight requirements, iterative and incremental development and communication (customers).

As a means of measuring agility, Williams et al. (2010) introduced a tool based on the Comparative Agility (CA) method created by co-authors Rubin and Cohn (available at <http://comparativeagility.com/>). The tool aims to help organizations determine their agility in relation to industry means, i.e., other

teams that have used the CA tool. The tool is based on a survey that evaluates agility in seven dimensions, each of which has its own characteristics. For each characteristic there are approximately four sentences that are evaluated by respondents on a Likert scale.

Buglione (2011) proposed a Light Maturity Model based on drivers that could be partially implemented in an organization. These drivers would be evaluated on a sheet using a typical 4-level ordinal scale adopted by most known models. The author suggested that the drivers evaluated could be based on dimensions or factors collected from models identified in the literature: testing, collective code ownership, collaboration, assurance/governance, simplicity, source code management, responsiveness to business, story formation, build process, requirements, build, shared responsibility, configuration management, geographical distribution, regulatory compliance, domain complexity, organization distribution, technical complexity, organizational complexity and enterprise discipline.

Recently, Soundararajan et al. (2012) described an approach for evaluating (a) the adequacy of an agile method for supporting defined goals; (b) the ability of an organization to implement this method; and (c) the effectiveness of the method. They proposed the OPP (Objectives, Principles and Practices) Framework, which is based on the concepts of adequacy (defining whether the method enables goals to be achieved), capability (the ability of the organization to provide an adequate environment) and effectiveness (the expected results are produced). The framework defines objectives and principles derived from the Agile Manifesto and a number of practices linked to these principles.

To measure whether agility supports business strategy, Kettunen (2012) proposed the Agility Profiler, a matrix with about 500 items that allow an organization to understand its agility needs as well how its current capabilities can be improved and obstacles removed. The profiler helps an organization identify drivers, goals, means, enablers and impediments to improving agility. According to the author, there is no such thing as a “good” or a “bad” profile as a profile is specific to the company strategy.

Schemes for measuring agility have also been proposed by the industrial sector, an example being the Nokia Test (Sutherland, 2007), which quickly identifies to what extent a team has adopted Scrum by assessing iterations, testing, specification, product-owner role, product backlog, estimates and the use of burndown charts.

Using these studies as a foundation, we identified different ways of characterizing or measuring agility. Layman et al. (2004) introduced some metrics to measure adherence to XP; Abbas et al. (2010) presented practices grouped into factors that could be correlated to success; Williams et al. (2010) used characteristics to measure agility; Buglione (2011) suggested drivers that could be evaluated in a Light Maturity Model; Soundararajan et al. (2012) defined practices linked to agile values; and Kettunen (2012) suggested measuring agility according to the business strategy.

To enable agile practitioners to evaluate agile practices and classify them in terms of maturity, we based our measurement instrument on a list of practices consolidated from these approaches. No distinction was made between agile methods that had different levels of emphasis on either project management or software development practices (Abrahamsson et al., 2003). Table 2 shows the practices and the author to whom they relate. The third and fourth columns show the corresponding statement and statement number in the survey measurement instrument.

This list consolidates the different characterizations of agile practices in the selected studies. However, as the authors of these studies apply these practices differently to characterize or measure agility in specific contexts, we lack evidence that the more practices a team adopts, the more agile it is. The next section explains how we used these practices in our study to redefine the concept of maturity in agile software development.

Table 1
Stage-based models for agile maturity.

Model	Author	Number of Levels	Highest maturity means. . .
Agile Maturity Map (AMM)	Packlick (2007)	5	. . . high performance teams that contribute to organizational learning and software engineering improvement
Sidky Agile Measurement Index (SAMI)	Sidky et al. (2007)	5	. . . sustaining agility after a collaborative, evolutionary, effective and adaptive environment is established
Agile Adoption and Improvement Model (AAIM)	Qumer and Henderson-Sellers (2008a)	6	. . . that after learning is established, the focus is on having quality production with minimal resources and keeping agility
Agile Maturity Model (AMM)	Patel and Ramachandran (2009)	5	. . . focusing on satisfaction of customers and development teams

3. Research approach

This section shows how we conducted our research (Section 3.1) and analyzed the resulting data (Sections 3.2 and 3.3) and includes a discussion on the validity of the results (Section 3.4)

3.1. Research question and procedures

Our research question in this study was “How do agile software development practitioners define maturity?” Our main objective was to identify how agile practices and the objectives of CMMI-DEV process areas are related to agile software development maturity. As the study sought to gain a preliminary insight into the topic (agile software development maturity), it is classified as an exploratory survey (Forza, 2002).

The survey took into account the issues suggested by Forza (2002) and Kitchenham et al. (2002) and used as its theoretical basis the conceptual framework developed following a review of the literature on agile software development maturity, as reported in Section 2. We searched for papers in the IEEE, ACM and ScienceDirect databases and included results from conferences and journals. This literature review allowed us to draw up a list of agile practices that served as a basis for developing the measurement instrument, i.e., the questionnaire (Table 2).

We collected the data by forwarding the questionnaire to Brazilian agile software development practitioners. To take part in the survey, respondents were required to have some experience in agile software development. The questionnaires were distributed in an online version using Qualtrics and the snowball technique (i.e., respondents sent the invitation to their own professional contacts) and in paper form to students on postgraduate software engineering courses. Although we included students in the sample, all of them were working with agile methods in companies. In all, the respondents represented thirty-three different Brazilian companies and four multinational companies that developed software primarily for their own use. Most of the respondents were developers and had up to three years of experience with agile methods, mainly Scrum. The respondents had also worked with the implementation of SPI models. Table 3 shows the respondents' profiles.

The questionnaire comprised two parts: in the first, the respondents evaluated 85 software development practices, and in the second they answered a single open-ended question.

In the first part of the questionnaire, the respondents had to evaluate and classify the agile practices identified in the literature (as shown in Table 2) on a 5-point Likert scale, ranging from 1 (“No Maturity”), through 2 (“Somewhat Mature”), 3 (“Mature”) and 4 (“Very Mature”), to 5 (“Very High Maturity”). Although our aim was to establish a definition of maturity in agile software development, we had to allow for the possibility that the definition would be the same as that associated with traditional CMMI-DEV. Hence,

the practices listed in the questionnaire included agile practices identified in the literature and CMMI-DEV process areas.

CMMI-DEV process areas had to be translated into practice-like statements to match the format of the other statements. For example, for the “Organizational Process Definition” process area, the purpose described in the CMMI-DEV document [“The purpose of Organizational Process Definition (OPD) is to establish and maintain a usable set of organizational process assets, work environment standards, and rules and guidelines for teams”] (SEI, 2010, p. 191) became statement X74: “Having defined process assets, work environment standards, and rules and guidelines”.

Following a suggestion by Forza (2002), we translated the practices identified in the literature into the empirical domain so that respondents could understand them. For example, “Regulatory compliance” became “Dealing easily with regulatory compliance”. Our respondents, when evaluating this statement, had to think about the question “In an agile environment, to what extent is it considered mature to deal easily with regulatory compliance?” and then answer this question on the 5-point Likert scale. It was made clear in the questionnaire in Portuguese that every statement refers to the ability of a team to perform the practice in question. Table 2 shows the translation to the empirical domain for all the agile practices.

In the second part of the questionnaire, in which respondents had to answer an open-ended question, we asked: “Based on your experience, what is maturity in agile software development?” A space approximately one-paragraph long was provided for respondents to write their definitions of maturity.

As our objective was to find out how practitioners define maturity in agile software development, we did not provide definitions of maturity. Instead, we wanted to allow the concept to emerge from the results. The respondents were also told that we did not intend to establish an association between the research results and the concept of maturity described in CMMI-DEV.

To improve the quality of the questionnaire, we conducted a pilot test with a group of fifteen software engineering researchers who were members of a software engineering research group in Brazil (masters or doctoral students) and had professional experience and research interests in software engineering. They tested both the printed and online form. Their suggestions helped to improve the format of the questionnaire and the presentation of the practices. The final version was made available online for ten days. We received fifty-one valid responses for the evaluation of the practices and forty-eighty for the open-ended question. The internal consistency of the responses (Cronbach's alpha) was 0.975.

3.2. Method used to analyze the maturity of practices

In view of the large number of maturity classifications (51 responses × 85 practices in the questionnaire) and as the practices

Table 2
Agile practices identified in the literature.

Author	Practice	Number	Related survey statement
Soundararajan et al. (2012)	Evolutionary requirements	X2	Allowing requirements to evolve during the project
	Refactoring	X13	Doing code refactoring
	Pair programming	X14	Doing pair programming
	Just in time	X47	Doing things when they have to be done, not before
	Self-organizing teams	X48	Self-organizing
	Physical setup reflecting agile philosophy	X60	Distributing physically so as to reflect agile philosophy
	Continuous delivery	X30	Delivering working software continuously
	Continuous feedback	X49	Giving continuous feedback
	Product backlog	X1	Using product backlog to define requirements
	Agile documentation	X50	Writing agile documentation
	Agile project estimation	X31	Making agile project estimates
	Retrospective meetings	X36	Holding retrospective meetings
	Client-driven iterations	X68	Allowing customer to drive iterations
	Appropriate distribution of expertise	X51	Distributing expertise on the team appropriately
	Iteration progress tracking and reporting	X37	Tracking and reporting iteration progress
Buglione (2011)	Collaboration	X43	Collaborating with team members
	Assurance/governance	X38	Integrating management activities directly into development tasks
	Simplicity	X44	Keeping work simple
	Source code management	X23	Managing source code
	Responsiveness to business	X67	Developing products that respond to business needs
	Story formation	X3	Using stories to define requirements
	Shared responsibility	X46	Sharing responsibility
	Configuration management	X22	Managing software configuration (version control)
	Geographical distribution	X57	Being geographically distributed (different cities or countries)
	Regulatory compliance	X59	Dealing easily with regulatory compliance
	Domain complexity	X63	Dealing easily with domain complexity
	Technical complexity	X64	Dealing easily with technical complexity
	Organizational complexity	X58	Dealing easily with organizational complexity
Abbas et al. (2010)	Enterprise discipline	X65	Dealing easily with enterprise discipline
	Traditional analysis	X8	Performing traditional systems analysis
	Database practice	X16	Being concerned about database architecture
	Agile quality assurance	X54	Doing agile quality assurance
	Communication (team)	X41	Communicating face-to-face daily
	Code analysis and inspection	X53	Analyzing and inspecting code
	Revision and lightweight testing	X52	Running lightweight tests and reviews
	Architecture and configuration	X11	Specifying software architecture
	Code standards	X15	Using code standards
	Lightweight requirements	X7	Defining lightweight requirements
	Iterative and incremental development	X28	Doing iterative and incremental development
Williams et al. (2010)	Communication (customers)	X66	Having the customer actively participate during the project
	Team composition	X61	Being multidisciplinary
	Focus	X40	Focusing on work (priorities do not change during iteration)
	Communication focus	X6	Eliciting requirements based on communication
	Emergence	X5	Allowing the emergence of requirements
	Technical design	X4	Doing technical design of requirements
	Planning levels	X25	Planning releases
	Critical variables	X32	Defining scope according to schedule
	Process tracking	X35	Holding daily progress tracking meetings
	Sources of dates and estimates	X33	Making estimates with the people who will do the work
	When the plan is drawn up	X26	Planning before and during the project
	Test driven development	X20	Doing test-driven development
	Continuous integration	X17	Doing continuous code integration
	Collective code ownership	X12	Using collective code ownership
	Management style	X45	Not losing autonomy when under pressure to meet deadlines
Layman et al. (2004)	Response to stress	X39	Responding to pressure by re-prioritizing or re-scoping instead of working overtime or adding people
	Title and salary alignment	X62	Encouraging a culture of working together as a team rather than individually
	Infrastructure	X55	Implementing development infrastructure that supports agility
	People	X56	Developing people's agility skills
	Timeboxes	X27	Using timeboxes in planning
	Team learning	X42	Questioning and learning from one another
	Short releases	X29	Making short software releases
	Onsite customer	X69	Customer being collocated
	Planning game	X24	Using the planning game
	Test LOC/source LOC	X21	Collecting test metrics
	Automated unit tests	X18	Running automated unit tests
	Customer acceptance tests	X19	Running user acceptance tests
	Simple design	X10	Using simple software design
	Sustainable pace	X34	Maintaining a sustainable pace (do minimum overtime)
	Metaphor	X9	Using metaphors to describe requirements

Table 3
Respondents' profiles.

		Number of respondents	Percentage
Role in their team	Developer	18	35
	System analyst	12	23
	Leader	8	16
	Software architect	5	10
	Test analyst	4	8
	Others	4	8
Experience with agile methods	From 1 to 3 years	27	53
	Less than 1 year	9	17
	From 4 to 6 years	8	16
	More than 6 years	7	14
Agile method	Scrum	36	70
	XP (Extreme Programming)	8	16
	Others (Kanban, customized methods)	7	14
Experience with SPI Models	CMMI-DEV	24	47
	MPS.BR ^a	11	22

^a Brazilian SPI model (Softex, 2012).

were sometimes similar or related to a specific topic, analysis of all the classifications individually would neither have been practical nor have allowed us to identify the general definition of maturity we were looking for. We therefore decided to use cluster analysis as a means of grouping practices according to the maturity classifications they received, thus reducing the amount of data so that we could take a broader view.

Cluster analysis allows natural groupings in data to be identified (Johnson and Wichern, 2007) and so satisfies our need to reduce the amount of data in order to identify a definition for maturity without losing information (Hair et al., 2006). In this type of analysis, the researcher does not define the number of groups a priori. In other words, the clustering of data is performed iteratively and the number of clusters is defined later, on the basis of similarities or distances between objects (Johnson and Wichern, 2007).

The clustering variable is the set of variables used to compare objects (Hair et al., 2006). In our study, the clustering variable is the classification given by the respondents on the 5-point Likert scale. Thus, by applying this method to the practices using this single variable, the practices could be grouped according to the maturity classifications assigned by the practitioners. According to Romesburg (2004), our application of cluster analysis fits the research goal of making a specific-purpose classification—identifying groups of practices that are similar according to the perceived level of maturity—and then extracting the definition of maturity from higher maturity clusters.

When applying cluster analysis, the researcher needs to address three main concerns (Hair et al., 2006): the fact that it is a descriptive, non-theoretical and non-inferential method; the fact that it will always create groupings even if they do not exist naturally; and the dependence on the variables used for similarity measurement.

With regard to the descriptive nature of cluster analysis, this was not a concern as we wished to carry out a descriptive analysis and were not looking for statistical inferences from groups based on the sample of respondents. On the contrary, we were interested in the answers in a compiled manner. Thus, the grouping was based on the practices, and not on the respondents. With respect to groups that do not exist naturally, this was not a concern. The maturity classification of practices was also analyzed individually, so that even “non-natural” clusters would not hinder our search for maturity classification. Finally, only one variable (maturity) was used for clustering, ensuring the unidimensionality of the underlying

variable and the inapplicability of multicollinearity effects (Balijepally et al., 2011).

Given the subjectivity involved in the application of cluster analysis, Balijepally et al. (2011) deem it important that details of the application method used be reported so that readers clearly understand the rationales for decisions. The following descriptions are based on the checklist provided by Balijepally et al.

Cluster variables. As mentioned previously, the only variable used for clustering was the maturity classification that respondents assigned to each practice, which was used to create clusters of practices. Therefore, variable standardization was not necessary and multicollinearity not possible. To measure similarity, we used the squared Euclidian distance, which is the recommended distance measure for Ward's clustering method (Hair et al., 2006).

Hierarchical clustering. Ward's clustering method was used. This method is recommended by Balijepally et al. (2011) because of its multi-measure approach to calculating similarity between clusters (Hair et al., 2006). One issue with this method is the treatment of outliers, which is not the case in this study because all practices remain interesting in our results, even when isolated in small clusters. The analysis was run in the SPSS statistical package.

Number of clusters. In cluster analysis, the final number of clusters is defined by the researcher based on the similarity of the content of each cluster (Hair et al., 2006). Using a dendrogram to analyze the clusters generated in an iterative process, we checked which practices were included in the clusters at each iteration and chose the solution with twenty clusters. An inductive approach was used (Balijepally et al., 2011) with verification of the inter-cluster separation using the Davies–Bouldin index, as discussed in the following section.

Cluster validation. A number of indexes can be used to validate the resulting clusters in a cluster analysis. In this study, we used the Davies–Bouldin index (Davies and Bouldin, 1979). This cluster separation measure is a function of the ratio of the sum of intra-cluster scatter to inter-cluster separation. The lower the resulting index, the more distant—and heterogeneous—the clusters are. We calculated the Davies–Bouldin index for five, ten, fifteen and twenty clusters, and the best index was obtained with the 5-cluster grouping. When analyzing the 5-cluster grouping of practices, we noted that the resulting clusters had around twenty practices each and that it was hard to name them because of the wide variety of practices in a single cluster. As this would make it difficult to formulate a definition of maturity, we chose to keep the 20-cluster solution.

We also performed a complementary validation of the 20-cluster solution with a sanity check to determine whether the practices inside each cluster were consistent in terms of the maturity classifications assigned by respondents. A sanity check based only on the similarities between practices in each cluster would be inconclusive because, for example, “Using the planning game” and “Establishing and maintaining plans that define project activities” are not meant to group in the same cluster just because both are related to planning. Thus, we calculated Cronbach's alpha to measure this internal consistency of each cluster. The resulting values ranged from 0.664 to 0.945. Only three out of the twenty clusters (“Lightweight requirements”, “Customer Presence” and “Caring about the Code”) had values under 0.7, which makes us confident that the group practices within each cluster were correlated in terms of maturity classifications (Bland and Altman, 1997).

After the clusters had been defined, we measured the extent to which individual practices and clusters were related to maturity. For each practice, we calculated the percentage of responses corresponding to classifications 1 or 2; 3; and 4 or 5. Using these results, we calculated the mean percentage for each cluster. For example, the “Lightweight Requirements” cluster has a mean percentage of 55.9% for classifications 1 or 2, 30.4% for classification 3 and 13.7% for classifications 4 or 5. These values were calculated as the means

Table 4
Clusters for agile software development maturity and percentage maturity classification assignments.

Cluster name	Practice number	Practice description	Percentage (practice) ^a			Total	Mean percentage (cluster) ^b		
			Low	Medium	High		Low	Medium	High
Collaboration	X41	Communicating face-to-face daily	19.6%	35.3%	45.1%	100%			
	X42	Questioning and learning from one another	7.8%	29.4%	62.7%	100%			
	X43	Collaborating with team members	7.8%	25.5%	66.7%	100%			
	X44	Keeping work simple	9.8%	35.3%	54.9%	100%			
	X45	Not losing autonomy when under pressure to meet deadlines	19.6%	29.4%	51.0%	100%			
	X46	Sharing responsibility	17.0%	23.5%	58.8%	100%			
Emerging requirements	X62	Encouraging a culture of working together as a team rather than individually	9.8%	31.4%	58.8%	100%	13.2%	30.0%	56.9%
	X2	Allowing requirements to evolve during the project	13.7%	35.3%	51.0%	100%			
	X5	Allowing the emergence of requirements	17.6%	27.5%	54.9%	100%	15.7%	31.4%	52.9%
Management of code and tests	X19	Running user acceptance tests	5.9%	35.3%	58.8%	100%			
	X21	Collecting test metrics	23.5%	31.4%	45.1%	100%			
	X22	Managing software configuration (version control)	11.8%	33.3%	54.9%	100%			
	X23	Managing source code	21.6%	31.4%	47.1%	100%			
	X25	Planning releases	15.7%	33.3%	51.0%	100%			
	X26	Planning before and during the project	15.7%	29.4%	54.9%	100%	15.7%	32.4%	52.0%
Caring about the solution	X15	Using code standards	13.7%	33.3%	52.9%	100%			
	X16	Being concerned about database architecture	11.8%	39.2%	49.0%	100%			
	X32	Defining scope according to schedule	23.5%	23.5%	52.9%	100%	16.3%	32.0%	51.6%
Test-driven development	X18	Running automated unit tests	21.6%	23.5%	54.9%	100%			
	X20	Doing test-driven development	29.4%	23.5%	47.1%	100%	25.5%	23.5%	51.0%
Sustainable self-organization	X12	Using collective code ownership	25.5%	31.4%	43.1%	100%			
	X17	Doing continuous code integration	15.7%	25.5%	58.8%	100%			
	X34	Maintaining a sustainable pace (do minimum overtime)	15.7%	25.5%	58.8%	100%			
	X39	Responding to pressure by re-prioritizing or re-scoping rather than working overtime or adding people	15.7%	31.4%	52.9%	100%			
	X48	Self-organizing	21.6%	33.3%	45.1%	100%			
	X49	Giving continuous feedback	17.6%	35.3%	47.1%	100%			
	X55	Implementing development infrastructure that supports agility	25.5%	27.5%	47.1%	100%			
	X56	Developing people's agility skills	21.6%	35.3%	43.1%	100%			
	X66	Having the customer actively participate during the project	11.8%	25.5%	62.7%	100%			
	X67	Developing products that respond to business needs	9.8%	41.2%	49.0%	100%	18.0%	31.2%	50.8%
	X75	Getting to know strengths and weaknesses and be able to plan and implement process improvements based on that	17.6%	35.3%	47.1%	100%			
Performance analysis	X76	Identifying gaps in performance and selecting and deploying improvements to close these gaps	15.7%	35.3%	49.0%	100%	16.7%	35.3%	48.0%
	X6	Eliciting requirements based on communication	11.8%	27.5%	60.8%	100%			
Simplicity	X10	Using simple software design	19.6%	47.1%	33.3%	100%	15.7%	37.3%	47.1%
	X35	Holding daily progress tracking meetings	29.4%	23.5%	47.1%	100%			
Meetings	X36	Holding retrospective meetings	23.5%	31.4%	45.1%	100%	26.5%	27.5%	46.1%
	X28	Doing iterative and incremental development	15.7%	43.1%	41.2%	100%			
Iterations	X29	Making short software releases	27.5%	31.4%	41.2%	100%			
	X30	Delivering working software continuously	17.6%	27.5%	54.9%	100%	20.3%	34.0%	45.8%
Manage requirements	X1	Using product backlog to define requirements	21.6%	37.3%	41.2%	100%			

Table 4
(Continued)

Cluster name	Practice number	Practice description	Percentage (practice) ^a			Total	Mean percentage (cluster) ^b		
			Low	Medium	High		Low	Medium	High
Traditional software process	X3	Using stories to define requirements	27.5%	39.2%	33.3%	100%			
	X11	Specifying software architecture	19.6%	35.3%	45.1%	100%			
	X27	Using timeboxes in planning	25.5%	33.3%	41.2%	100%			
	X33	Making estimates with the people who will do the work	17.6%	33.3%	49.0%	100%			
	X61	Being multidisciplinary	13.7%	43.1%	43.1%	100%			
	X70	Identifying causes of problems and taking actions to prevent them in the future	13.7%	35.3%	51.0%	100%			
	X78	Developing people's skills and knowledge so they can perform their roles effectively and efficiently	17.6%	27.5%	54.9%	100%	19.6%	35.5%	44.9%
	X58	Dealing easily with organizational complexity	19.6%	41.2%	39.2%	100%			
	X59	Dealing easily with regulatory compliance	15.7%	45.1%	39.2%	100%			
	X63	Dealing easily with domain complexity	13.7%	47.1%	39.2%	100%			
	X64	Dealing easily with technical complexity	13.7%	43.1%	43.1%	100%			
	X65	Dealing easily with enterprise discipline	9.8%	45.1%	45.1%	100%			
	X74	Having defined process assets, work environment standards, and rules and guidelines	25.5%	29.4%	45.1%	100%			
	X79	Establishing and maintaining plans that define project activities	23.5%	37.3%	39.2%	100%			
	X80	Evaluating processes and work products objectively and addressing non-compliance issues	21.6%	41.2%	37.3%	100%			
	X82	Formally eliciting, analyzing, and validating requirements for the product and stakeholders	21.6%	45.1%	33.3%	100%			
	X83	Planning and invoking risk handling activities as needed across the life of the project	23.5%	45.1%	31.4%	100%			
	X84	Managing the acquisition of products and services from suppliers	25.5%	47.1%	27.5%	100%			
	X85	Maintaining alignment between requirements and the project's plans and work products	17.6%	51.0%	31.4%	100%	19.3%	43.1%	37.6%
Agile project management	X24	Using the planning game	41.2%	25.5%	33.3%	100%			
	X31	Making agile project estimates	27.5%	39.2%	33.3%	100%			
	X50	Writing agile documentation	39.2%	23.5%	37.3%	100%	35.9%	29.4%	34.6%
Project monitoring	X37	Tracking and reporting iteration progress	25.5%	43.1%	31.4%	100%			
	X38	Integrating management activities directly into development tasks	23.5%	45.1%	31.4%	100%			
	X71	Analyzing possible decisions using a formal evaluation process that evaluates identified alternatives against established criteria	29.4%	35.3%	35.3%	100%			
	X72	Managing projects according to an integrated and defined process	27.5%	29.4%	43.1%	100%			
	X73	Collecting metrics that are used to support management information needs	27.5%	41.2%	31.4%	100%			
	X77	Having a quantitative understanding (metrics-based) of processes	27.5%	37.3%	35.3%	100%			
	X81	Managing projects with measures and analytic techniques	41.2%	29.4%	29.4%	100%	28.9%	37.3%	33.9%
	X57	Being geographically distributed (different cities or countries)	39.2%	27.5%	33.3%	100%			
Physical distribution	X60	Distributing physically so as to reflect agile philosophy	29.4%	37.3%	33.3%	100%	34.3%	32.4%	33.3%
	X13	Doing code refactoring	23.5%	27.5%	49.0%	100%			
Agile coding	X14	Doing pair programming	35.3%	33.3%	31.4%	100%			
	X40	Focusing on work (priorities do not change during iteration)	29.4%	45.1%	25.5%	100%			
	X47	Doing things when they have to be done, not before	35.3%	37.3%	27.5%	100%			
	X54	Doing agile quality assurance	37.3%	31.4%	31.4%	100%	32.2%	34.9%	32.9%

Table 4
(Continued)

Cluster name	Practice number	Practice description	Percentage (practice) ^a			Total	Mean percentage (cluster) ^b		
			Low	Medium	High		Low	Medium	High
Customer presence	X68	Allowing customer to drive iterations	43.1%	25.5%	31.4%	100%	47.1%	28.4%	24.5%
	X69	Customer being collocated	51.0%	31.4%	17.6%	100%			
Caring about the code	X4	Doing technical design of requirements	31.4%	49.0%	19.6%	100%	30.9%	45.6%	23.5%
	X51	Distributing expertise on the team appropriately	17.6%	47.1%	35.3%	100%			
	X52	Running lightweight tests and reviews	39.2%	52.9%	7.8%	100%			
	X53	Analyzing and inspecting code	35.3%	33.3%	31.4%	100%			
Lightweight requirements	X7	Defining lightweight requirements	43.1%	35.3%	21.6%	100%	55.9%	30.4%	13.7%
	X9	Using metaphors to describe requirements	68.6%	25.5%	5.9%	100%			
Traditional analysis	X8	Performing traditional systems analysis	56.9%	29.4%	13.7%	100%	56.9%	29.4%	13.7%

^a This column is divided into three subcolumns: “Low” shows the percentage of responses that associated this practice with level 1 (“No Maturity”) or 2 (“Somewhat Mature”); “Medium” shows the percentage of responses that associated this practice with level 3 (“Mature”); and “High”, the percentage that associated the practice with level 4 (“Very Mature”) or 5 (“Very High Maturity”).

^b This column is divided into three subcolumns: “Low” shows the mean percentage of responses that associated the practices in each cluster with level 1 (“No Maturity”) or 2 (“Somewhat Mature”); “Medium” shows the mean percentage of responses that associated the practices in each cluster with level 3 (“Mature”); and “High”, the mean percentage that associated the practices in each cluster with level 4 (“Very Mature”) or 5 (“Very High Maturity”).

of the corresponding percentages for the practices X7—“Defining lightweight requirements” and X9—“Using metaphors to describe requirements” (as shown in Table 4).

3.3. Method used to analyze the open-ended question

As we were concerned with ensuring greater confidence in the definition of maturity that emerged from the questionnaire, triangulation was performed using two analysis methods (Bryman, 2012): the quantitative approach for the analysis of responses described in Section 3.2 and qualitative analysis of the open-ended question in the questionnaire.

The texts of the responses to the open-ended question were analyzed using a content analysis technique (Bardin, 2011), which involves systematically going through data and finding meaning according to hypotheses or objectives. Content analysis is performed in three steps: pre-analysis, data exploration and treatment of the results.

Bardin (2011) explains that pre-analysis is the phase when a plan is drawn up for the analysis. This phase comprises three main activities: choosing documents, formulating hypotheses and objectives and building indicators to support the final interpretation. In this study, the documents were the survey responses, which were organized with the aid of NVivo software. The objective was to find out how practitioners define maturity in agile software development. The indicator interpreted was the frequency of occurrence of certain elements in the responses. Analyzing the frequency of specific elements that appeared in the responses allowed us to identify the main concepts that define maturity in agile software development in practice.

Data exploration is the step when text coding is performed. This is the process in which the raw text is transformed into a representation that will clarify a concept (Bardin, 2011). The recording unit used was the practitioner response (about a paragraph in length). As codes were identified in the response, NVivo kept the original references and counted each occurrence. After we analyzed all the responses, a table of frequencies was created. We then classified the codes into categories and subcategories so that we could infer any relationships between them. The classification criterion used for this purpose was intended to identify whether the element was related to development practices, process, team, stakeholders, management or outcomes.

The last step is the treatment of the results to find meaning. We analyzed the frequencies of codes and categories and created a diagram that consolidated the elements associated with maturity in agile software development. These data are presented in Section 5.

3.4. Validity of the research

Here we address two main concerns regarding the validity of our study: the small sample size and the use of the snowball technique for sampling.

The first is clearly a problem of statistical significance. The data were collected from a sample of individuals in a population whose size we cannot estimate, and the clusters generated and concepts identified in the content analysis may be valid only for our sample rather than for the whole population (Lee and Mohareji, 2012).

We are aware that much of the rigor of research is based on the concept of statistical significance (Lee and Mohareji, 2012). However, when Hair et al. (2006) suggest that the sample size in cluster analysis must be large enough to provide representations of groups, they emphasize the need to find *practical relevance* in the clusters results. This is also an issue in qualitative studies, of which our content analysis is an example, as statistical generalization is not usually possible and researchers need to pursue analytical generalization (Sjøberg et al., 2008).

Indeed, Kitchenham et al. (2002) consider practical relevance in scientific studies to be different from statistical significance. For the former, a study must generate findings that matter to practitioners, which is especially important if one considers that software engineering is an applied discipline and that the results of any research must be of interest to the software industry (Sjøberg et al., 2008). Consequently, studies need to address not only statistical significance, but also effect sizes when they use statistical analysis such as ANOVA, *t* test, *R* and *R_c* (Kirk, 2001; Thompson, 2002); and, no less important, they need to address the applicability of the results of research in practice. This issue has been discussed in a number of studies in the context of management science (Gopinath and Hoffman, 1995; Nicolai and Seidl, 2010) and medicine (Thompson, 2002).

Nicolai and Seidl (2010) argue that the concept of relevance has yet to be well defined and propose a taxonomy for relevance in management studies. As our study addresses not only agile

software development practices, but also those of project management, we decided to borrow their taxonomy. They differentiate between knowledge that has “conceptual relevance”, “instrumental relevance” and “legitimative relevance” (Nicolai and Seidl, 2010, p. 1263).

We consider the knowledge that our findings generate to have “conceptual relevance”. It is scientific knowledge that affects how we perceive a decision situation or modifies our understanding of a decision situation (Nicolai and Seidl, 2010). The outcome of our paper is a definition of agile software development maturity that modifies the current established concept of maturity in software engineering. It is a new linguistic construct, which has “the potential to change the way we think and communicate about our world and, by extension, about our decision situations” (Nicolai and Seidl, 2010, p. 1267).

Lee and Mohareji (2012) suggest that one way to identify practical relevance in research is to conduct further empirical studies on the same subject, as we did. We recently conducted a survey of experienced agile practitioners (Fontana et al., 2014), which revealed that they see maturity from a subjective viewpoint and believe that an Agile Maturity Model would be useful in the industry if it addressed the context-specific characteristics of teams. This confirms the practical relevance of the findings presented here. Furthermore, the practical relevance of our findings is corroborated in recent studies that assess Agile Maturity Models, confirming the need for a new approach to maturity in agility (Leppänen, 2013; Ozcan-Top and Demirörs, 2013; Schweigert et al., 2012).

Thus, although the concept of agile software development maturity presented in this study may have been generated from a small sample size, the scientific rigor underlying the study and the conceptual relevance of the findings ensure that the scientific contribution made by the study is valid. Nevertheless, as with any scientific research, further studies are required to confirm or refute our results.

The second concern is the snowball sampling technique. This method is applicable when a sampling frame cannot be drawn from the population (Bryman, 2012), as is the case here. Although agile methods have yet to gain widespread acceptance in Brazil, their use is on the rise (Melo et al., 2013), making the snowball sampling technique a viable form of contacting agile practitioners.

4. Quantitative data analysis

In this section we present the results of the analysis of the maturity classifications that respondents assigned to the 85 practices in our questionnaire. Table 4 shows how the cluster analysis algorithm grouped the practices.

We named each cluster based on the characteristics of the practices contained in it. Table 4 also shows the percentage of respondents that associated each practice with a maturity classification. The percentage of respondents who answered that the maturity for a specific practice was 1 or 2 (“No Maturity” or “Somewhat Mature”) is shown in the column “Low”; the percentage that answered 3 (“Mature”) is shown in the column “Medium”; and the percentage answering 4 or 5 (“Very Mature” or “Very High Maturity”), is shown in the column “High”. The percentage and mean percentage of respondents for each maturity classification is given for each practice and cluster, respectively, and the highest percentage is highlighted in bold. Clusters are presented in decreasing order of percentage of assignments to the highest maturity classification.

The lowest-maturity cluster of practices is the only one with one practice. This outlier emerged from the cluster analysis because the algorithm did not find another practice with a maturity classification similar to it. We called this cluster “Traditional Analysis”,

and 56.9% of the respondents considered it low maturity. Next, the “Lightweight Requirements” cluster contains two practices related to agile requirements elicitation, which 55.9% of respondents associated with lower maturity, a value similar to that for the “Traditional Analysis” cluster.

Next in increasing order of maturity as perceived by practitioners is the “Caring about the Code” cluster (considered medium maturity by 45.6% of respondents). This cluster contains four practices that represent techniques for improving source code quality and one that relates to the appropriate distribution of expertise in the team. The “Customer Presence” cluster represents customers being close to the team and driving work and was associated with low maturity by 47.1% of respondents.

Next, the “Agile Coding” cluster, which groups coding practices such as refactoring and pair programming with focused work and quality assurance, had very similar percentages for the low-, medium- and high-maturity classifications. The highest percentage corresponded to the medium-maturity classification (34.9% of respondents). The “Physical Distribution” cluster represents practices related to how a team is physically distributed and was considered low-maturity by 34.3% of practitioners, a value also very close to the percentages of respondents who rated it medium- and high-maturity.

“Project Monitoring” is a medium-maturity cluster (according to 37.3% of respondents). It was given this name because the practices grouped together in it relate to project tracking and metrics collection activities for understanding project status. Next, 35.9% of practitioners assigned a low-maturity classification to the “Agile Project Management” cluster, which contains the planning game, agile-like project estimates and agile-like documentation.

“Traditional Software Process” had a medium-maturity classification (43.1% of respondents). Any practices that describe dealing with any kind of complexity or the formalization and standardization of work were grouped in this cluster. Most are derived from CMMI-DEV process areas.

The “Manage Requirements” cluster is the first group of practices for which the percentage of respondents (44.9%) that considered it high-maturity was greater than the percentages that considered it low- or medium-maturity. The practices in the cluster relate to requirements, planning and people. The cluster combines practices that favor the management of requirements throughout the development process. Next is the “Iterations” cluster, which has very similar classifications and was considered high-maturity by 45.8% of practitioners. It brings together practices related to iterative development and continuous delivery.

The “Meetings” and “Simplicity” clusters were associated with a high level of maturity by 46.1% and 47.1% of respondents, respectively. The “Performance Analysis” cluster, which groups practices deriving from CMMI-DEV, received a high-maturity classification from 48% of the practitioners. It represents practices related to knowing one’s own performance, strengths and weaknesses.

The next six clusters were assigned high maturity classifications by over half of the practitioners. We called the cluster classified as high-maturity by 50.8% of the respondents “Sustainable Self-organization” because the practices grouped in this cluster relate to creating the environment for agility, team self-organization and having the customer involved in the project with results that add value to business. Next came the “Test-Driven Development” cluster, which groups technical practices associated with test automation (51%) and the “Caring about the Solution” cluster, with practices related to architecture and defining a reasonable scope (51.6%).

The top three clusters related to higher maturity in agile software development are “Management of Code and Tests”, “Emerging Requirements” and “Collaboration”. The first was designated as higher-maturity by 52% of respondents and includes most of the

practices related to agile testing and coding and two related to planning before and during the project, as well as test metrics. As described previously, project metrics were assigned to the “Project Monitoring” cluster, which was not as closely associated with high maturity. Next, the “Emerging Requirements” cluster was considered higher-maturity by 52.9% of respondents and has two practices related to allowing changes in requirements.

The highest maturity cluster was “Collaboration”, which was assigned a high-maturity classification by 56.9% of practitioners. It was given this name because the practices it combines are associated with collaboration, communication, learning and interaction, i.e., different aspects of the way a team works.

During the quantitative analysis of data, it became clear that practitioners perceive practices differently when considering maturity: as shown in Table 4, some agile software development practices are associated with a lack of maturity and others with a high level of maturity. The resulting clusters and the analysis of the frequencies with which maturity classifications were assigned to practices revealed that the concepts associated with higher maturity were Management of Code and Tests, Emerging Requirements and Collaboration. The practices derived from CMMI-DEV process areas (statements X70–X85) were classified as mature but not high-maturity. For example, the “Traditional Software Process” cluster was classified as medium-maturity by 43.1% of respondents and high-maturity by 37.6%.

To triangulate the results, we analyzed the answers to the open-ended question given by practitioners, in which they described agile software development maturity according to their experience. The results of the qualitative analysis of these data are presented in the next section.

5. Qualitative analysis

The codes created during content analysis (Bardin, 2011) are the practices or achievements—which we will call concepts—that respondents used to define maturity in agile software development. The fifty-two key concepts that we found in the analysis are listed in Table 5 and classified into categories and subcategories. When cited in the text, concepts, categories and subcategories are shown in *italics* to ease identification.

This section is divided into three subsections: 5.1 Key concepts, in which we explain the codes identified; 5.2 Numbers, where we give the frequencies with which the concepts occurred in the responses; and 5.3 Relationships, in which we relate categories to a definition of agile software development maturity.

5.1. Key concepts

The concepts in the *Development Practices* category in Table 5 are agile methods for software development defined in XP (Beck and Andres, 2004), as well as some traditional software engineering activities (*Configuration Management*, *Development Standards* and *Software Testing*).

Concepts that are similar to generic goals in CMMI-DEV were classified as *Process*. These appeared in responses in relation to agile values together with concepts such as the *Application of Agile Practices* and the *Standardization of Agile Practices*.

In the *Team* category, a number of concepts related to *Knowledge*, *Behavior*, *Communication* and *Experience* emerged. For the subcategory *Knowledge*, respondents pointed out that it is important to be aware of the specific business, project and technology, as well as having a *Trained Team* and *Keeping Lessons Learned*. For *Behavior*, to be mature a team has to practice *Collaboration* and *Commitment*, and it must make an *Effort to keep practices in use*. *Self-organization* is another behavioral need. One respondent

Table 5

Key concepts that define agile software development maturity (based on the responses in the survey).

Category	Subcategory	Concept
Development practices	–	Configuration management
		Continuous delivery of working software Development standards Sufficient software documentation Pair programming Refactoring Software testing Test-driven development
Process	–	Application of agile practices Continuous improvement Definition of tools and methods Metrics-based improvement Process institutionalization Standardization of agile practices Use of tools and methods
Team	Knowledge	Keep lessons learned Knowledge of the customer's business Knowledge of the project Knowledge of the technology Trained team Collaboration Commitment Making an effort to keep practices in use Self-organization Understand customers
	Behavior	Communication within the team Communication with customers
	Communication	Expertise in agile practices Time spent working with agile
	Experience	
Stakeholders	–	Agile process acceptance Definition of business priorities Stakeholders information
Management	–	Definition of goals Process management Process metrics Project planning Project tracking
Outcomes	For management	Efficiency Fewer defects Less effort Less rework Less waste Precise estimates Predictability Productivity Repetition of results Delivery on time Effectiveness Flexibility Generate value for the customer Product quality Short delivery time
	For customer	

defined it as “[...] when the Scrum Master becomes a secretary with administrative functions only”, i.e., with no need to control activities. *Understand Customers* is also a concept related to *Behavior* within the *Team* because the team must make a continuous effort to *Understand Customers* so that they can meet customers' expectations. For *Communication*, practitioners pointed out the need to continuously interact by direct communication inside and outside

the team and, for the *Experience* category, reported the need to have some background in agile methods.

The *Stakeholders* category represents the role stakeholders play in agile maturity. For external stakeholders, *Agile Process Acceptance* and *Business Priorities Definition* are necessary as guidelines for work. For both internal and external stakeholders, it is important that *Stakeholder Information* is shared. This means that everybody is aware of what is happening in the project and shares vocabulary and knowledge about it. Interestingly, one respondent wrote that maturity in agile software development is “an almost utopian situation in which customer and team reach a high-level understanding with respect to deliveries, dates and specifications”.

We found that project and process management are also related to maturity in agile software development. In the *Management* category, practitioners mentioned the importance of planning and tracking the project, managing processes by using metrics and defining goals.

A number of responses described maturity in terms of outcomes, i.e., the results that a mature team generates. We placed all the concepts we found with this definition in the category *Outcomes*. The responses indicated that these outcomes may be delivered *For Management*, for whom the benefits of maturity may include *Efficiency*, *Less Rework*, *Less Waste*, *Less Effort*, *Fewer Defects*, *Repetition of Results*, *Precise Estimates*, *Predictability* and *Productivity*. The outcomes may also be perceived by Customers (*For Customers* subcategory), who benefit from the added value, *Short Delivery Time*, *Delivery on Time*, *Effectiveness*, *Flexibility* and *Product Quality*.

One may argue that requirements elicitation and management, the entry points of the software lifecycle, are missing from our analysis. In fact, the practitioners did not point out specific requirements practices as characteristics of maturity in agile software development. One possible explanation is that their answers focused on the outcome of good requirements elicitation and management, i.e., *Generating Value for the Customer*.

5.2. Numbers

We were able to infer what the most relevant concepts for maturity in agile software development were by analyzing the frequency with which the various concepts occurred. Fig. 1 shows the concepts that appeared three or more times in the responses. Those that appeared the most frequently (six, seven or eight times) relate to the use of processes, methods and tools for applying agility to generate high-quality and value-adding software for customers. The agile values *Flexibility*, *Continuous Delivery of Working Software*, *Short Delivery Time* and *Collaboration* were the next most frequent, with five and three occurrences, along with *Efficiency* and *Effectiveness*, *Stakeholders Information*, *Understanding Customer* and *Continuous Improvement*.

Because categories help to simplify raw data (Bardin, 2011) and group similar elements, analysis of the frequency of occurrence at category level allows additional interpretations of the data to be made. Fig. 2 shows that maturity in agile software development is mainly perceived in terms of *Outcomes*, followed by *Process* and *Team*. *Development Practices*, *Stakeholders* and *Management* are secondary categories and occur less frequently.

5.3. Relationships

The analysis of the answers to the open-ended question allowed us to identify the relationships between the categories, as shown in Fig. 3. The *Team* category is the one that relates to all others. A mature, agile *Team* focuses on *Communicating* and has *Experience*,



Fig. 1. Frequency of occurrence of concepts.

agile *Behavior* and *Knowledge* about the specific business, project and technology and lessons learned. The *Team* must be aligned with the *Stakeholders*, and vice versa, through intense communication to gain agile process acceptance, share knowledge and set business priorities. A mature team applies *Processes* that define and standardize work in agile ways. *Management* directs goals, makes plans and tracks what is going on. Finally, it is the *Team* that generates the *Outcomes* perceived when maturity is reached. These outcomes are classified as relevant for either management or customers.

Content analysis of the responses to the open-ended question about what constitutes maturity in agile development revealed that practitioners in agile software development define maturity as the generation of specific outcomes with defined processes built with agility. The team plays a central role—indeed, one that is as important as the processes themselves—in allowing maturity to be achieved.

6. Discussion

We found that practitioners define maturity in agile software development in terms of agile community values rather than process definition and control, as CMMI-DEV and ISO/IEC 15504 do.

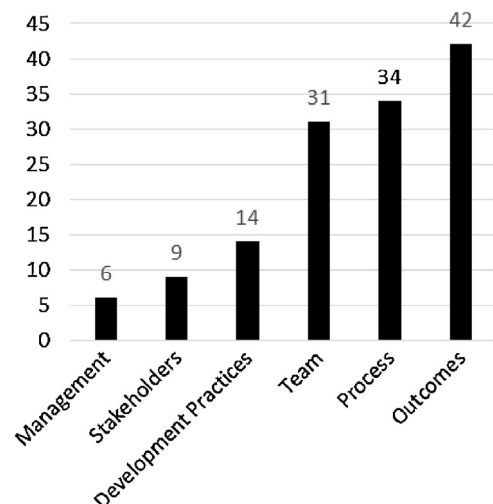


Fig. 2. Frequency of occurrence of categories.

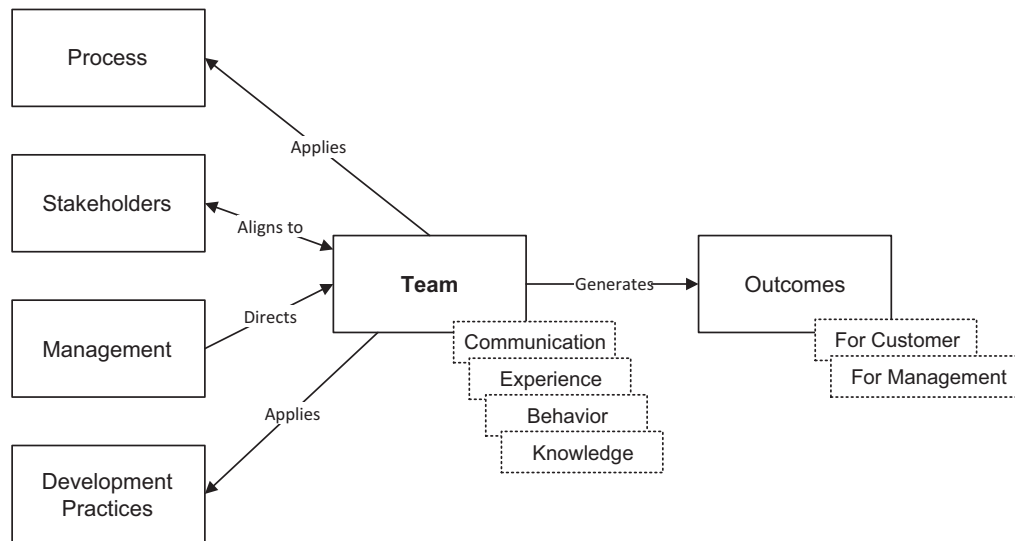


Fig. 3. Relationships between the concepts that define maturity in agile software development (based on survey responses).

The highest-maturity clusters of practices and the concepts that emerged from the practitioners' definitions enabled us to propose the following definition of agile software development maturity:

Maturity in agile software development means having an experienced team that:

- collaborates on projects by communicating and being committed;
- cares about customers and software quality;
- allows requirements to change;
- shares knowledge;
- manages source code and tests using tools, methods and metrics supported by infrastructure appropriate for agility;
- self-organizes at a sustainable pace;
- standardizes and continuously improves agile practices; and
- generates perceived outcomes for customers and management.

Our quantitative analysis of the classification of the eighty-five practices showed that higher-maturity practices are those that support Sustainable Self-organization, Test-driven Development, Caring about the Solution, Management of Code and Tests, Emerging Requirements and, especially, Collaboration. These results are supported by the qualitative analysis of the answers to the open-ended question. The practitioners' concepts of maturity revealed that this is perceived mainly in the outcomes generated by the team for both management and customers. To generate these outcomes, Team and Processes play an equally important role: the process is defined and standardized by a team that collaborates and self-organizes. Table 6 gives details of some studies that address each of the points we included in our definition of maturity in agile software development.

Our results run counter to current initiatives to implement CMMI-DEV requirements in order to develop maturity in agile environments in that they show that although processes are still necessary for agile maturity, they must be based on agile practices, i.e., they must leave room for flexibility and agility. In the quantitative analysis, the objectives of the CMMI-DEV process were still classified as mature, but team capabilities, such as collaboration, and requirements flexibility were considered more important for maturity.

Packlick (2007), Sidky et al. (2007), Qumer and Henderson-Sellers (2008a) and Patel and Ramachandran (2009) do not address maturity concepts, but maturity models. If the characteristics of the highest maturity levels in the models are analyzed, it will be

seen that each describes maturity in quite a different way. Although the models all consider agile values in the maturing process, there remains a lack of consensus among these authors regarding the definition of maturity. Our study steps back and proposes a definition of the concept first of all, thus providing a basis for future studies to develop a road map to maturity.

Maier et al. (2012) point out that the current concept of maturity is based on a codified business process to enable organizational improvement. Our contribution with this study is to demonstrate that there is more to agile software development than process definition. If higher-maturity teams should be flexible and self-organized, accommodate emerging requirements and deliver within a short timescale, how can this be accomplished with a predefined set of practices and levels? Sidky et al. (2007), Kettunen (2012), Schweigert et al. (2012) and Fontana et al. (2014) presented evidence that agile practitioners do not believe in prescriptive models for agile maturity. It is our view that to develop guides that will help improve agile software development there is a need to shift responsibility for developing maturity from processes alone and take a broader and perhaps more subjective view (Maier et al., 2012) in which people and interaction play an all-important role.

Both our study and studies by other authors (Leppänen, 2013; Ozcan-Top and Demirörs, 2013; Schweigert et al., 2012) have identified that current Agile Maturity Models still differ in their underlying structure, indicating that there is an ongoing debate in the field. Further studies on agile software development maturity are necessary to refine this new maturity concept and discover how agile teams develop capabilities on their way to maturity.

7. Conclusion

This was an exploratory study to discover whether agile maturity is the same as that defined in current SPI models. We conducted a survey with Brazilian agile practitioners and analyzed quantitative and qualitative data. Our findings allowed us to propose a definition for agile software development maturity that includes not only the definition and improvement of processes, but also some more subjective capabilities such as collaboration, communication, commitment, care, sharing and self-organization.

However, our findings are subject to some limitations. The first is that the definition of maturity is based on respondents' perceptions

Table 6

Studies that address elements of the definition of maturity.

Maturity in agile software development means having an experienced team that	Studies that discuss the issue
Collaborates on projects by communicating and being committed	McHugh et al. (2012) found that using practices such as sprint/iteration planning, daily stand-up and sprint/iteration retrospective help the team to work more cohesively, thereby improving trust. Aarnink and Kruithof (2012) claim that an important characteristic of agile software development is a high level of involvement of business members and that factors such as communication, partnership and competence indicate high added value in agile software development organizations. Carvalho (2013) presents an integrated communication framework summarizing the role of individual and organizational perspectives at different maturity levels
Cares for customers and software quality	Melo et al. (2013) noted that agile methods improve customer satisfaction by delivering value frequently and that quality is a perceived benefit. Huo et al. (2004) identified how agile methods ensure quality during the development process, and Manjunath et al. (2013) showed how Lean and Scrum can be combined to achieve high-quality products
Allows requirements to change	Gupta et al. (2013) proposed a dynamic for requirements reprioritization, a task that is required to allow requirements to change. Bjarnason et al. (2012) provided details of the factors involved in overscoping and requirements management
Shares knowledge	Ryan and O'Connor (2013) described the processes of acquiring and sharing tacit knowledge in software development. Tiwana (2008) presented interesting conclusions about how knowledge integration relates to strong ties and bridging ties in project alliances
Manages source code and tests by using tools, methods and metrics supported by infrastructure appropriate for agility	Janzen and Saiedian (2005) described test-driven development and the issues involved. Sjøberg et al. (2012) presented some agile methods and metrics for quantifying their performance in practice. A thorough review on software metrics can be found in Kitchenham (2010)
Self-organizes at a sustainable pace	Hoda et al. (2013) conducted a grounded theory study to discover how agile teams self-organize and identified specific generic roles that can be observed in any agile team. According to Vidgen and Wang (2006), self-organization and other agile practices are part of complex adaptive systems theory
Standardizes and continuously improves agile practices	Standardization and continuous improvement are usually related to software process definition. Some interesting findings related to software process are included in Coleman and O'Connor (2008) and Adolph et al. (2012). Both studies emphasize practical aspects of the software process and do not consider models or methods
Generates perceived outcomes for customers and management	Dybå and Dingsøyr (2008) analyzed thirty-six empirical studies and discussed, among other issues, customers', developers' and students' perceptions of agile methods. In the practical domain, Melo et al. (2013) found that the main perceived outcomes of the implementation of agile methods were productivity, the ability to manage changing priorities, team morale, a simplified development process, quality, customer satisfaction, job satisfaction and project visibility

of the maturity of the practices listed in the measurement instrument rather than on observation of real projects. The practitioners gave their own interpretations based on their experience, which is a characteristic of opinion-based surveys (Kitchenham and Pfleeger, 2008). We therefore used the analysis of the open-ended question to minimize the effects of this limitation on the interpretation of the practices listed in the questionnaire. The second limitation is that as the findings are based on the analysis of data obtained exclusively from Brazilian agile practitioners, they can only be generalized for this agile adoption profile, which is described in Melo et al. (2013).

Nonetheless, our findings have practical relevance (Kitchenham et al., 2002; Nicolai and Seidl, 2010) as they provide a starting point for researchers and practitioners to consider the finding that an agile software development team needs more than process definition to become mature. In other words, we need to address how to create maturity models and guides that will support the development of capabilities rather than merely prescribe a list of processes.

The objective of this study was not to propose a maturity model. If there is concern regarding the road map to maturity, the clusters that resulted from our cluster analysis could be a starting point for future studies to investigate to what extent these clusters could serve as levels or areas for agile software process improvement. In fact, our findings emphasize the need to reconsider the definition of maturity in agile software development and to shift from an increasing focus on process to an increasing focus on people.

References

- Aarnink, A., Kruithof, G., 2012. Contribution of agile software development methods to business – IT alignment non-profit organizations. *Commun. IIMA* 12 (2), Available at: <http://goo.gl/AK9VzV>
- Abbas, N., Gravell, A.M., Wills, G.B., 2010. Using factor analysis to generate clusters of agile practices – a guide for agile process improvement. In: 2010 Agile Conference, pp. 11–20. <http://dx.doi.org/10.1109/AGILE.2010.15>
- Abrahamsson, P., Warsta, J., Siponen, M., Ronkainen, J., 2003. New directions on agile methods: a comparative analysis ICSE'03. In: Proceedings of the 25th International Conference on Software Engineering, 3–10 May, pp. 244–254. <http://dx.doi.org/10.1109/ICSE.2003.1201204>
- Adolph, S., Krutchen, P., Hall, W., 2012. Reconciling perspectives: a grounded theory of how people manage the process of software development. *J. Syst. Softw.* 85 (June (6)), 1269–1286. <http://dx.doi.org/10.1016/j.jss.2012.01.059>
- Anderson, D.J., 2005. Stretching agile to fit CMMI level 3 – the story of creating MSF for CMMI process improvement at microsoft corporation. In: Proceedings of the Agile Conference (ADC'05), 24–29 July, pp. 193–201. <http://dx.doi.org/10.1109/ADC.2005.42>
- Baker, S.W., 2006. Formalizing agility. Part 2: How an agile organization embraced the CMMI. In: Proceedings of the AGILE 2006 Conference, 23–28 July, pp. 146–154. <http://dx.doi.org/10.1109/AGILE.2006.30>
- Balijepally, V., Mangalaraj, G., Iyengar, K., 2011. Are we wielding this hammer correctly? A reflective review of the application of cluster analysis in information systems research. *J. Assoc. Inform. Syst.* 12 (May (5)), 375–413, Available at: <http://goo.gl/Df6y88>
- Bardin, L., 2011. *Análise de Conteúdo*. Edições, pp. 70.
- Beck, K., et al., 2001. Agile Manifesto, Available at: <http://agilemanifesto.org/> (accessed May 2013).
- Beck, K., Andres, C., 2004. *Extreme Programming Explained: Embrace Change*. Pearson Education Inc.
- Bhasin, S., 2012. Quality assurance in agile – a study towards achieving excellence. In: Agile India 2012, 17–19 February, pp. 64–67. <http://dx.doi.org/10.1109/AgileIndia.2012.18>
- Bjarnason, E., Wnuk, K., Regnell, B., 2012. Are you biting off more than you can chew? A case study on causes and effects of overscoping in large-scale

- software engineering. *Inform. Softw. Technol.* 54 (October (10)), 1107–1124, <http://dx.doi.org/10.1016/j.infsof.2012.04.006>.
- Bland, J.M., Altman, D.G., 1997. Statistics notes: Cronbach's alpha. *BMJ* 314, 572, <http://dx.doi.org/10.1136/bmj.314.7080.572>.
- Boehm, B., Turner, R., 2003. Observations on balancing discipline and agility. In: *Proceedings of the Agile Development Conference*, 25–28 June, pp. 32–39, <http://dx.doi.org/10.1109/ADC.2003.1231450>.
- Bryman, A., 2012. *Social Research Methods*, 4th ed. Oxford University Press, New York.
- Buglione, L., 2011. Light Maturity Models (LMM): an agile application Profes'11. In: *Proceedings of the 12th International Conference on Product Focused Software Development and Process Improvement*, pp. 57–61, <http://dx.doi.org/10.1145/2181101.2181115>.
- Bustard, D., Wilkie, G., Greer, D., 2013. The maturation of agile software development principles and practice: observations on successive industrial studies in 2010 and 2012. In: *20th Annual IEEE International Conference and Workshops on the Engineering of Computer Based Systems (EBCS)*, 22–24 April, pp. 139–146, <http://dx.doi.org/10.1109/ECBS.2013.11>.
- Carvalho, M., 2013. An investigation of the role of communication in IT projects. *Int. J. Oper. Prod. Manage.* 34 (1), 36–64, <http://dx.doi.org/10.1108/IJOPM-11-2011-0439>.
- Cohan, S., Glazer, H., 2009. An agile development team's quest for CMMI maturity level 5. In: *Agile Conference 2009*, 24–29 August, pp. 201–206, <http://dx.doi.org/10.1109/AGILE.2009.24>.
- Coleman, G., O'Connor, R., 2008. Investigating software process in practice: a grounded theory perspective. *J. Syst. Softw.* 81 (5), 772–784, <http://dx.doi.org/10.1016/j.jss.2007.07.027>.
- Davies, D.L., Bouldin, D.W., 1979. A cluster separation measure. *IEEE Trans. Pattern Anal. PAMI-1* (2), 224–227, <http://dx.doi.org/10.1109/TPAMI.1979.4766909>.
- Dingsøyr, T., Nerur, S., Balijepally, V., Moe, N.B., 2012. A decade of agile methodologies: towards explaining agile software development. *J. Syst. Softw.* 85 (6), 1213–1221, <http://dx.doi.org/10.1016/j.jss.2012.02.033>.
- Dybå, T., Dingsøyr, T., 2008. Empirical studies of agile software development: a systematic review. *Inform. Softw. Technol.* 50 (9–10), 833–859, <http://dx.doi.org/10.1016/j.infsof.2008.01.006>.
- Fontana, R.M., Reinehr, S., Malucelli, A., 2014. Maturing in agile: what is it about? In: *Proceedings of the 15th International Conference, XP 2014*, Rome, Italy, May 26–30, pp. 94–109, http://dx.doi.org/10.1007/978-3-319-06862-6_7.
- Forza, C., 2002. Survey research in operations management: a process-based perspective. *Int. J. Oper. Prod. Manage.* 22 (2), 152–194, <http://dx.doi.org/10.1108/01443570210414310>.
- Gopinath, C., Hoffman, R.C., 1995. The relevance of strategy research: practitioner and academic viewpoints. *J. Manage. Stud.* 32 (September (5)), 575–594, <http://dx.doi.org/10.1111/j.1467-6486.1995.tb00789.x>.
- Gupta, V., Chauhan, D.S., Dutta, K., Gupta, C., 2013. Requirement reprioritization: a multilayered dynamic approach. *Int. J. Softw. Eng. Appl.* 7 (5), 55–64, <http://dx.doi.org/10.14257/ijseia.2013.7.5.06>.
- Hair, J., Black, B., Babin, B., Anderson, R.E., Tatham, R.L., 2006. *Multivariate Data Analysis*, 6th ed. Pearson Prentice Hall, Englewood Cliffs, NJ.
- Hoda, R., Noble, J., Marshall, S., 2013. Self-organizing roles on agile software development teams. *IEEE Trans. Softw. Eng.* 39 (3), 422–444, <http://dx.doi.org/10.1109/TSE.2012.30>.
- Huo, M., Verner, J., Zhu, L., Babar, M.A., 2004. Software quality and agile methods. In: *Proceedings of the 28th Annual International Computer Software and Applications Conference (COMPSAC'04)*, 28–30 September, pp. 520–525, <http://dx.doi.org/10.1109/COMPSAC.2004.1342889>.
- ISO/IEC: 15504-1, 2004. *Information Technology – Process Assessment. Part 1: Concepts and Vocabulary*. ISO/IEC, Geneva, Switzerland, Available at: <http://goo.gl/DZJfU5>.
- Jakobsen, C.R., Johnson, K.A., 2008. Mature agile with a twist of CMMI. In: *Agile Conference 2008*, 4–8 August, pp. 212–217, <http://dx.doi.org/10.1109/Agile.2008.10>.
- Janzen, D., Saiedian, H., 2005. Test-driven development: concepts, taxonomy and future direction. *Computer* 38 (September (9)), 43–50, <http://dx.doi.org/10.1109/MC.2005.314>.
- Johnson, R.A., Wichern, D.W., 2007. *Applied Multivariate Statistical Analysis*, 6th ed. Person Prentice Hall, NJ.
- Kettunen, P., 2012. Systematizing software development agility: towards an enterprise capability improvement framework. *J. Enterp. Transform.* 2 (2), 81–104, <http://dx.doi.org/10.1080/19488289.2012.664610>.
- Kirk, Roger E., 2001. Promoting good statistical practices: some suggestions. *Educ. Psychol. Meas.* 61 (2), 213–218, <http://dx.doi.org/10.1177/00131640121971185>.
- Kitchenham, B., 2010. What's up with software metrics? – a preliminary mapping study. *J. Syst. Softw.* 83 (January (1)), 37–51, <http://dx.doi.org/10.1016/j.jss.2009.06.041>.
- Kitchenham, B.A., Pfleeger, S.L., 2008. Personal opinion surveys. In: Shull, F., Singer, J., Sjöberg, D. (Eds.), *Guide to Advanced Empirical Software Engineering*. Springer, London, pp. 63–92, http://dx.doi.org/10.1007/978-1-84800-044-5_3.
- Kitchenham, B.A., Pfleeger, S.L., Pickard, L.M., Jones, P.W., Hoaglin, D.C., Emam, K.E., Rosenberg, J., 2002. Preliminary guidelines for empirical research in software engineering. *IEEE Trans. Softw. Eng.* 28 (August (8)), 721–734, <http://dx.doi.org/10.1109/TSE.2002.1027796>.
- Kohlegger, M., Maier, R., Thalmann, S., 2009. Understanding maturity models results of a structured content analysis. In: *Proceedings of the I-KNOW'09 and I-SEMANTICS'09*, 2–4 September 2009, Available at: <http://goo.gl/hnw7uh>.
- Layman, L., Williams, L., Cunningham, L., 2004. Motivations and measurements in an agile case study QUTE-SWAP'04. In: *Proceedings of the 2004 Workshop on Quantitative Techniques for Software Agile Process*, pp. 14–24, <http://dx.doi.org/10.1145/1151433.1151436>.
- Lee, A.S., Mohareji, K., 2012. Linking relevance to practical significance. In: *Proceedings of the 45th Hawaii International Conference on System Sciences*, Maui, HI, 4–7 January, pp. 5234–5240, <http://dx.doi.org/10.1109/HICSS.2012.416>.
- Leppänen, M., 2013. A comparative analysis of agile maturity models. In: Pooley, R. (Ed.), *Information Systems Development: Reflections, Challenges and New Directions*. Springer Science+Business Media, New York, pp. 329–343, http://dx.doi.org/10.1007/978-1-4614-1951-5_27.
- Lukasiewicz, K., Miler, J., 2012. Improving agility and discipline of software development with the Scrum and CMMI. *IET Softw.* 6 (5), 416–422, <http://dx.doi.org/10.1049/iet-sen.2011.0193>.
- Maier, A.M., Moutrie, J., Clarkson, J., 2012. Assessing organizational capabilities: reviewing and guiding the development of maturity grids. *IEEE Trans. Eng. Manage.* 59 (February (1)), 138–159, <http://dx.doi.org/10.1109/TEM.2010.2077289>.
- Manjunath, K.N., Jagadeesh, J., Yogeesh, M., 2013. Achieving quality product in a long term software product development in healthcare application using Lean and Agile principles. In: *Proceedings of 2013 International Multi-Conference on the Automation, Computing, Communication, Control and Compressed Sensing (iMac4s)*, 22–23 March, pp. 26–34, <http://dx.doi.org/10.1109/iMac4s.2013.6526379>.
- McHugh, O., Conboy, K., Lang, M., 2012. Agile practices: the impact on trust in software project teams. *IEEE Softw.* 29 (May/June (3)), 71–76, <http://dx.doi.org/10.1109/MS.2011.118>.
- Melo, C.O., Santos, V., Katayama, E., Corbucci, H., Prikladnicki, R., Goldman, A., Kon, F., 2013. The evolution of agile software development in Brazil. *J. Braz. Comput. Soc.* 19 (4), 523–552, <http://dx.doi.org/10.1007/s13173-013-0114-x>.
- Nicolai, A., Seidl, D., 2010. That's relevant! Different forms of practical relevance in management science. *Org. Stud.* 31 (9–10), 1257–1285, <http://dx.doi.org/10.1177/0170840610374401>.
- Ozcan-Top, O., Demirörs, O., 2013. Assessment of agile maturity models: a multiple case study. In: *Software Process Improvement and Capability Determination, 13th International Conference, SPICE 2013*, Bremen, Germany, June 4–6, pp. 130–141, http://dx.doi.org/10.1007/978-3-642-38833-0_12.
- Packlick, J., 2007. The agility maturity map – a goal oriented approach to agile improvement. In: *Agile Conference*, 13–17 August, pp. 266–271, <http://dx.doi.org/10.1109/AGILE.2007.55>.
- Patel, C., Ramachandran, M., 2009. Agile Maturity Model (AMM): a software process improvement framework for agile software development practices. *Int. J. Softw. Eng.* 2 (1), 3–28, Available at: <http://goo.gl/FGe0eE>.
- Paulk, M., 2001. Extreme programming from a CMM perspective. *IEEE Softw.* 18 (6), 19–26, <http://dx.doi.org/10.1109/52.965798>.
- Qumer, A., Henderson-Sellers, B., 2008a. A framework to support the evaluation, adoption and improvement of agile methods in practice. *J. Syst. Softw.* 81 (11), 1899–1919, <http://dx.doi.org/10.1016/j.jss.2007.12.806>.
- Qumer, A., Henderson-Sellers, B., 2008b. An evaluation of the degree of agility in six agile methods and its applicability for method engineering. *Inform. Softw. Technol.* 50 (4), 280–295, <http://dx.doi.org/10.1016/j.infsof.2007.02.002>.
- Romesburg, C., 2004. *Cluster Analysis for Researchers*. Lulu Press, NC, Available at: <http://goo.gl/PxTkf4>.
- Ryan, S., O'Connor, R.V., 2013. Acquiring and sharing tacit knowledge in software development teams: an empirical study. *Inform. Softw. Technol.* 55 (September (9)), 1614–1624, <http://dx.doi.org/10.1016/j.infsof.2013.02.013>.
- Schweigert, T., Nevalainen, R., Vohwinkel, D., Korsaa, M., Biro, M., 2012. Agile maturity model: oxymoron or the next level of understanding. In: Mas, A., et al. (Eds.), *SPICE 2012*, 29–31 May, pp. 289–294, http://dx.doi.org/10.1007/978-3-642-30439-2_34.
- SEI, 2010. *CMMI for Development Version 1.3*. Technical Report, Available at: <http://goo.gl/kjzxy>.
- Siakas, K.V., Siakas, E., 2007. The agile professional culture: a source of agile quality. *Softw. Process Improv. Pract.* 12 (6), 597–610, <http://dx.doi.org/10.1002/spip.344>.
- Sidky, A., Arthur, J., Bohner, S., 2007. A disciplined approach to adopting agile practices: the agile adoption framework. *Innov. Syst. Softw. Eng.* 3 (3), 203–216, <http://dx.doi.org/10.1007/s11334-007-0026-z>.
- Sjöberg, D.I.K., Dybå, T., Anda, B.C.D., Hanny, J.E., 2008. Building theories in software engineering. In: Shull, F., Singer, J., Sjöberg, D. (Eds.), *Guide to Advanced Empirical Software Engineering*. Springer, London, <http://dx.doi.org/10.1007/978-1-84800-044-5>.
- Sjöberg, D.I.K., Johnsen, A., Solberg, J., 2012. Quantifying the effect of using Kanban versus Scrum: a case study. *IEEE Softw.* 29 (September–October (5)), 47–53, <http://dx.doi.org/10.1109/MS.2012.110>.
- Softex, 2012. *Guia Geral MPS de Software*, Available at: <http://goo.gl/rwYrys>.
- Soundararajan, S., Arthur, J.D., Balci, O., 2012. A methodology for assessing agile software development methods. In: *Agile Conference 2012*, 13–17 August, pp. 51–54, <http://dx.doi.org/10.1109/Agile.2012.24>.
- Sutherland, J., 2007. *Nokia Test*, Available at: <http://jeffsutherland.com/nokiatest.pdf>.
- Sutherland, J., Jakobsen, C.R., Johnson, K., 2007. Scrum and CMMI level 5: the magic potion for code warriors. In: *Agile Conference 2007*, 13–17 August, pp. 272–278, <http://dx.doi.org/10.1109/AGILE.2007.52>.
- Thompson, B., 2002. “Statistical,” “Practical” and “Clinical”: how many kinds of significance do counselors need to consider? *J. Couns. Dev.* 80 (Winter (1)), 64–71, <http://dx.doi.org/10.1002/j.1556-6678.2002.tb00167.x>.

- Tiwana, A., 2008. Do bridging ties complement strong ties? An empirical examination of alliance ambidexterity. *Strat. Manage. J.* 29 (3), 251–272, <http://dx.doi.org/10.1002/smj.666>.
- Vidgen, R., Wang, X., 2006. Organizing for agility: a complex adaptive systems perspective on agile software development process. In: Ljunberg, J., Andersson, M. (Eds.), *Proceedings of the Fourteenth European Conference on Information Systems*. Gothenburg, Sweden, pp. 1316–1327. Available at: <http://goo.gl/b5oGLX>
- Williams, L., Krebs, W., Layman, L., Antón, A., Abrahamsson, P., 2004. Toward a framework for evaluating extreme programming. In: *8th International Conference on Empirical Assessment in Software Engineering (EASE 04)*, pp. 11–20, <http://dx.doi.org/10.1049/ic:20040394>.
- Williams, L., Rubin, K., Cohn, M., 2010. Driving process improvement via comparative agility assessment. In: *Agile Conference 2010*, 9–13 August, pp. 3–10, <http://dx.doi.org/10.1109/AGILE.2010.12>.

Rafaela Mantovani Fontana has a Bachelor's degree in Computer Science and a Master's Degree in Systems and Production Engineering. She is a doctoral student at the Pontifical Catholic University of Paraná and a professor at the Federal University of Paraná. Her research interests include agile software development methods, project management, software process improvement, software quality and complexity theory applied to management.

Isabela Mantovani Fontana is a doctoral student in Industrial Engineering at the University of São Paulo. She has a Master's degree in Design from the Federal University of Paraná, a Bachelor's degree in Industrial Design—Product

Design from the Pontifical Catholic University of Paraná and an MBA degree in Business Management (FGV-ISAE). She has research experience in the fields of collaborative design, product-service systems and the mass customization process.

Paula Andrea da Rosa Garbuio has a Bachelor's Degree in Civil Engineering from the Federal University of Paraná and a Master's Degree in Systems and Production Engineering from the Pontifical Catholic University of Paraná. She is a Lean Six Sigma Black Belt certified professional and has experience in transportation logistics, with a focus on quality management.

Sheila Reinehr has a Bachelor's Degree in Mechanical Engineering, a Master's Degree in Informatics and a Doctorate in Engineering. She is a professor at the Pontifical Catholic University of Paraná and an experienced researcher in the following areas of computer science: software engineering, software process improvement, software quality, project management, software product lines and metrics.

Andreia Malucelli has a Bachelor's Degree in Informatics, a Master's Degree in Electrical Engineering and a Doctorate in Electrical and Computing Engineering. She is a professor at the Pontifical Catholic University of Paraná and an experienced researcher in the following areas of computer science: software engineering, artificial intelligence, organizational learning, ontologies, multiagent systems and healthcare information systems.